

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных
задачах»
на тему «Оценка времени выполнения программ»

Выполнили:

студенты группы 25BBB4

Сулова А.В.

Саченкова О.А.

Приняли:

Юрова О.В.,

Деев М.В.

Пенза 2026

1. Цель

Изучить методы оценки времени выполнения программ, определить вычислительную сложность алгоритмов и экспериментально исследовать зависимость времени их выполнения от размера входных данных.

2. Задания

Задание 1: вычислить порядок сложности программы (О-символику); оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000; построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2: оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива; оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел; оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел; оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую; оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

3. Пояснительный текст к программам

Задание 1

Основная часть программы выполняет умножение двух квадратных матриц размера $N \times N$. Для этого используются три вложенных цикла по индексам i , r и j . Каждый цикл выполняется N раз, поэтому внутренняя операция умножения и сложения выполняется N^3 раз. Поэтому сложность операции умножения матриц составляет $O(N^3)$. Заполнение исходных матриц выполняется с помощью двух вложенных циклов и имеет сложность $O(N^2)$. Общая сложность программы составляет: $O(N^2) + O(N^3) = O(N^3)$.

Теоретическая оценка времени (условно взято $T(1000) = 0,2$ с):

Количество элементов	Оценочное время (с)
100	0,0002
200	0,0008
400	0,0064
1000	0,2
2000	1,6
4000	12,8
10000	200

Программа выполняет умножение двух квадратных матриц размера $N \times N$ и измеряет время выполнения этого вычисления. Сначала создаются три статические матрицы a , b и c . Матрицы a и b заполняются случайными числами от 1 до 100, а матрица c заполняется нулями. Затем с помощью трёх вложенных циклов вычисляется произведение матриц: $C=A*B$. Для каждого элемента матрицы C последовательно суммируются произведения соответствующих элементов строк матрицы A и столбцов матрицы B . Время выполнения основной части программы измеряется с помощью функции `clock()` и выводится в секундах.

Результаты работы программы:

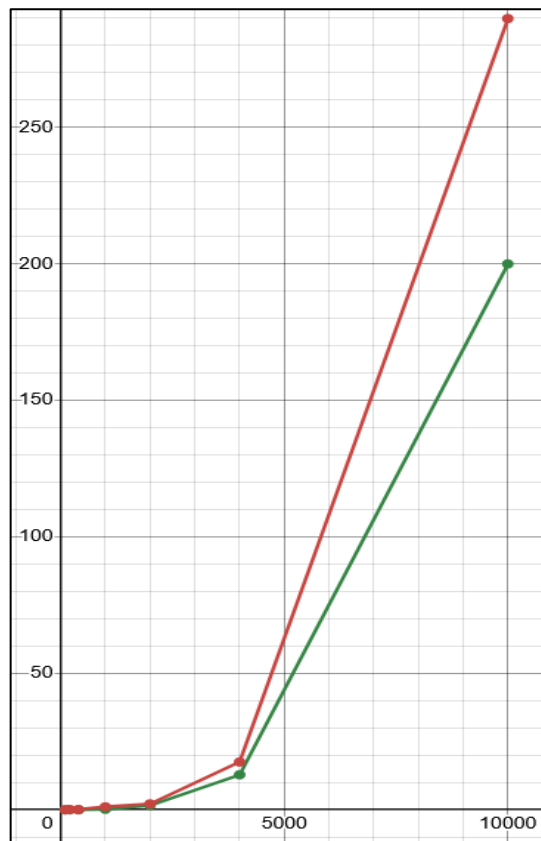
```

N = 100      N = 200      N = 400
Time: 0.003000 seconds  time: 0.016000 seconds  time: 0.154000 seconds
N = 1000     N = 2000     N = 4000
time: 0.130000 seconds  time: 2.147000 seconds  Time: 17.538000 seconds
N = 10000
Time: 289.841000 seconds

```

График зависимости времени выполнения программы от размера матриц (N_1/t_1 – график с практическими значениями, N_2/T_2 – с оценочными):

N_1	 t_1	N_2	 t_2
100	0.003	100	0.0002
200	0.16	200	0.0008
400	0.154	400	0.0064
1000	1.13	1000	0.2
2000	2.147	2000	1.6
4000	17.538	4000	12.8
10000	289.841	10000	200



Сравнивая с теоретической оценкой, значения времени, полученные с помощью программы, больше.

Задание 2

Программа предназначена для сравнения времени работы алгоритмов сортировки целочисленного массива: сортировки Шелла (shell), быстрой сортировки (qs) и стандартной функции qsort из библиотеки C. Для каждого алгоритма создаётся отдельная копия исходного массива, чтобы все алгоритмы работали с одинаковыми исходными данными. Время сортировки измеряется с помощью функции clock(). После сортировки дополнительно проверяется, действительно ли массив отсортирован.

Испытания проводятся на четырёх типах массивов размером 10000 элементов: случайном, возрастающем, убывающем, а также массиве, первая половина которого упорядочена по возрастанию, а вторая — по убыванию. Полученные времена выполнения и результаты проверки сортировки выводятся на экран. В итоге программа позволяет сравнить производительность реализованных алгоритмов и стандартной функции qsort на различных исходных данных.

Результат работы программы (count = 10000):

```

random
shell: 0.012000 sec | sorted: yes
quick: 0.003000 sec | sorted: yes
qsort: 0.004000 sec | sorted: yes

increasing
shell: 0.001000 sec | sorted: yes
quick: 0.000000 sec | sorted: yes
qsort: 0.004000 sec | sorted: yes

decreasing
shell: 0.015000 sec | sorted: yes
quick: 0.000000 sec | sorted: yes
qsort: 0.001000 sec | sorted: yes

increasing + decreasing
shell: 0.006000 sec | sorted: yes
quick: 0.019000 sec | sorted: yes
qsort: 0.002000 sec | sorted: yes

```

Результат работы программы (count = 100000):

```

random
shell: 0.565000 sec | sorted: yes
quick: 0.013000 sec | sorted: yes
qsort: 0.021000 sec | sorted: yes

increasing
shell: 0.001000 sec | sorted: yes
quick: 0.003000 sec | sorted: yes
qsort: 0.008000 sec | sorted: yes

decreasing
shell: 0.983000 sec | sorted: yes
quick: 0.001000 sec | sorted: yes
qsort: 0.007000 sec | sorted: yes

increasing + decreasing
shell: 0.402000 sec | sorted: yes
quick: 1.543000 sec | sorted: yes
qsort: 0.027000 sec | sorted: yes

```

Самая быстрая сортировка:

Случайный набор значений: quick;

Возрастающая последовательность чисел: 10000 элементов - quick, 100000 элементов – shell (вывод - скорости сортировок могут меняться относительно друг друга в зависимости от количества элементов массива);

Убывающая последовательность чисел: quick;

Первая половина возрастает, вторая убывает: qsort.

4. Псевдокод

Задание 1

НАЧАЛО

Задать размер матриц $N = 1000$

Создать матрицы A , B и C размером $N \times N$

Инициализировать генератор случайных чисел

Для i от 0 до $N - 1$

 Для j от 0 до $N - 1$

 Записать в $A[i][j]$ случайное число от 1 до 100

 Записать в $B[i][j]$ случайное число от 1 до 100

 Записать в $C[i][j]$ значение 0

 Конец цикла j

Конец цикла i

Запомнить время начала вычислений

Для i от 0 до $N - 1$

 Для r от 0 до $N - 1$

$t = A[i][r]$

 Для j от 0 до $N - 1$

$C[i][j] = C[i][j] + t * B[r][j]$

 Конец цикла j

 Конец цикла r

Конец цикла i

Запомнить время окончания вычислений

Вычислить время выполнения:

время = время окончания - время начала

Вывести размер матрицы N

Вывести время выполнения

КОНЕЦ

Задание 2

НАЧАЛО

Задать количество элементов массива count = 10000

Выделить память для исходного массива original

Если память не выделена

Вывести сообщение об ошибке

Завершить программу

Инициализировать генератор случайных чисел

// 1. Случайный массив

Для i от 0 до count - 1

Записать в original[i] случайное число

Конец цикла

Провести тестирование для массива "random"

// 2. Возрастающий массив

Для i от 0 до $\text{count} - 1$

$\text{original}[i] = i$

Конец цикла

Провести тестирование для массива "increasing"

// 3. Убывающий массив

Для i от 0 до $\text{count} - 1$

$\text{original}[i] = \text{count} - i$

Конец цикла

Провести тестирование для массива "decreasing"

// 4. Массив: первая половина возрастающая,

// вторая половина убывающая

Для i от 0 до $\text{count} / 2 - 1$

$\text{original}[i] = i$

Конец цикла

Для i от $\text{count} / 2$ до $\text{count} - 1$

$\text{original}[i] = \text{count} - i$

Конец цикла

Провести тестирование для массива

"increasing + decreasing"

Освободить память исходного массива

КОНЕЦ

Псевдокод процедуры тестирования

Процедура TEST_SET(original, count, name)

Выделить память для трёх массивов:

А, В, С

Если память не выделена

Вывести ошибку

Освободить выделенную память

Завершить процедуру

Скопировать original в А

Скопировать original в В

Скопировать original в С

Запустить таймер

Отсортировать А методом Шелла

Остановить таймер

Сохранить время

Запустить таймер

Отсортировать В быстрой сортировкой

Остановить таймер

Сохранить время

Запустить таймер

Отсортировать С стандартной функцией qsort

Остановить таймер

Сохранить время

Проверить, отсортирован ли А

Проверить, отсортирован ли В

Проверить, отсортирован ли С

Вывести название набора данных

Вывести время работы сортировки Шелла

и результат проверки

Вывести время работы быстрой сортировки

и результат проверки

Вывести время работы qsort

и результат проверки

Освободить память А, В, С

Конец процедуры

5. Вывод

Изучили методы оценки времени выполнения программ, определили вычислительную сложность алгоритмов и экспериментально исследовали зависимость времени их выполнения от размера входных данных.